Шпаргалка к защите: Alg5.cpp

Преподаватель показывает на код и спрашивает «что делает?» и «как делает?». Здесь — короткие ответы, которые можно сказать своими словами.

Программа целиком

Что делает программа?

Сравнивает скорость трёх сортировок: вставками, слиянием и стандартной `sort`. Для каждого размера массива делает 10 случайных массивов, сортирует их всеми тремя способами, засекает время и считает среднее. Потом то же самое на массивах из отсортированных кусков. Всё это делается два раза: для массивов `int` и для массивов `double`. Результаты печатает на экран и сохраняет в файл results.csv.

Почему все функции шаблонные?

Чтобы один и тот же код работал и с `int`, и с `double`. Шаблонные сортировки, заполнение, проверка, печать и замер. Нешаблонная только `printHeader` — она печатает шапку таблицы и с массивами не работает.

insertionSort — сортировка вставками

```
template <typename T>
void insertionSort(T arr[], int size) {
    for (int i = 1; i < size; i++) {
        T n = arr[i];
        int j = i - 1;
        while (j >= 0 && n < arr[j]) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = n;
    }
}
```

Что делает?

Сортирует массив вставками.

Как делает?

Идёт по массиву слева направо. Берёт очередной элемент, сдвигает вправо все элементы перед ним, которые больше него, и ставит его в освободившееся место. Левая часть массива всегда уже отсортирована.

Если показывают на строку	Что ответить
`for (int i = 1; ...)`	С единицы, потому что первый элемент один и уже отсортирован
`T n = arr[i];`	Запоминаем элемент, который вставляем
`int j = i - 1;`	Начинаем смотреть с соседа слева
`while (j >= 0 && n < arr[j])`	Пока не дошли до начала и слева элемент больше
`arr[j + 1] = arr[j];`	Сдвигаем больший элемент на одну позицию вправо
`j--;`	Идём дальше влево
`arr[j + 1] = n;`	Ставим элемент на найденное место

Сложность?

O(n^2): два вложенных цикла, каждый до n.

mergeSort — сортировка слиянием

```
template <typename T>
void mergeSort(T arr[], int size) {
    T* buf = new T[size];
    mergeSortRange(arr, buf, 0, size);
    delete[] buf;
}
```

Что делает?

Создаёт вспомогательный массив, вызывает сортировку слиянием и потом удаляет этот массив.

Как делает?

`new` выделяет буфер, `mergeSortRange` сортирует весь массив от 0 до size, `delete[]` освобождает память.

Зачем буфер?

При слиянии двух половин результат нужно куда-то складывать. В сам массив нельзя — затрём ещё не взятые элементы.

mergeSortRange — рекурсивная часть слияния

```
template <typename T>
void mergeSortRange(T arr[], T buf[], int left, int right) {
    if (right - left <= 1) return;

    int center = (right - left) / 2 + left;

    mergeSortRange(arr, buf, left, center);
    mergeSortRange(arr, buf, center, right);

    int i = left;
    int j = center;

    for (int k = left; k < right; k++) {
        if (j == right || (i < center && arr[i] <= arr[j])) {
            buf[k] = arr[i++];
        }
        else buf[k] = arr[j++];
    }

    for (int k = left; k < right; k++) {
        arr[k] = buf[k];
    }
}
```

Что делает?

Сортирует кусок массива от `left` до `right`.

Как делает?

Делит кусок пополам, каждую половину сортирует сама собой (рекурсия), потом сливает две отсортированные половины в одну: каждый раз берёт меньший из двух первых элементов.

Если показывают на строку	Что ответить
`if (right - left <= 1) return;`	Если в куске 1 элемент или пусто — он уже отсортирован, выходим. Без этого рекурсия была бы бесконечной
`int center = (right - left) / 2 + left;`	Находим середину
`mergeSortRange(arr, buf, left, center);`	Сортируем левую половину
`mergeSortRange(arr, buf, center, right);`	Сортируем правую половину
`int i = left; int j = center;`	`i` — начало левой половины, `j` — начало правой
`for (int k = left; k < right; k++)`	Заполняем буфер по одному элементу
`if (j == right || (i < center && arr[i] <= arr[j]))`	Берём из левой половины, если правая закончилась или левый элемент не больше правого
`buf[k] = arr[i++];`	Кладём элемент в буфер и сдвигаем `i` дальше
`else buf[k] = arr[j++];`	Иначе берём из правой половины
`for (...) arr[k] = buf[k];`	Копируем результат из буфера обратно в массив

Сложность?

O(n log n): делим пополам — это log n уровней, на каждом уровне проходим все n элементов.

stdSort

```
template <typename T>
void stdSort(T arr[], int size) {
    sort(arr, arr + size);
}
```

Что делает?

Вызывает стандартную `sort`.

Зачем отдельная функция?

Чтобы она выглядела так же, как наши две, — (массив, размер). Тогда все три можно передавать в одну функцию замера.

fillRandom

```
template <typename T>
void fillRandom(T arr[], int size) {
    for (int i = 0; i < size; i++) {
        arr[i] = (T)rand() + (T)rand() / RAND_MAX;
    }
}
```

Что делает?

Заполняет массив случайными числами.

Как делает?

`(T)rand()` — целая часть, `(T)rand() / RAND_MAX` — дробная от 0 до 1. Для double получается число вроде 12345.678. Для int деление целое, дробная часть отбрасывается.

Что значит `(T)`?

Приведение типа: превращаем число в тот тип, с которым работает шаблон (int или double).

fillFragments

```
template <typename T>
void fillFragments(T arr[], int size) {
    fillRandom(arr, size);
    int len = size / PARTS;
    for (int p = 0; p < PARTS; p++) {
        int left = p * len;
        int right = (p == PARTS - 1) ? size : left + len;
        sort(arr + left, arr + right);
    }
}
```

Что делает?

Делает массив из 10 отсортированных кусков.

Как делает?

Сначала заполняет случайно, потом делит на 10 частей и каждую часть сортирует.

Что значит `(p == PARTS - 1) ? size : left + len`?

Если это последний кусок — он идёт до конца массива, иначе его длина len. Так последний кусок забирает остаток, если размер не делится на 10.

isSorted и printArray

```
template <typename T>
bool isSorted(T arr[], int size) {
    for (int i = 1; i < size; i++) {
        if (arr[i] < arr[i - 1]) return false;
    }
    return true;
}
```

Что делает?

Проверяет, что массив отсортирован.

Как делает?

Сравнивает каждый элемент с предыдущим. Если где-то предыдущий больше — возвращает false.

Зачем?

Чтобы убедиться, что сортировка правильная.

```
template <typename T>
void printArray(T arr[], int size) {
    cout << "[ ";
    for (int i = 0; i < size; i++) {
        cout << arr[i];
        if (i < size - 1) cout << ", ";
    }
    cout << " ]" << endl;
}
```

Что делает printArray?

Печатает массив в квадратных скобках через запятую.

measure — замер одной сортировки

```
template <typename T>
double measure(void (*sortFunc)(T[], int), T src[], T arr[], int size) {
    for (int i = 0; i < size; i++) arr[i] = src[i];

    auto start = chrono::steady_clock::now();
    sortFunc(arr, size);
    auto end = chrono::steady_clock::now();

    if (!isSorted(arr, size)) {
        cout << "ERROR: array is not sorted!" << endl;
        exit(1);
    }
    return chrono::duration<double, milli>(end - start).count();
}
```

Что делает?

Меряет время одной сортировки в миллисекундах.

Как делает?

Копирует исходный массив, засекает время, сортирует, засекает время снова и возвращает разницу. Потом проверяет, что массив отсортирован.

Что могут спросить	Что ответить
`void (*sortFunc)(T[], int)` — что это?	Указатель на функцию. Сюда передаём, какой сортировкой сортировать
Зачем копировать в `arr`?	Чтобы все три сортировки получили одинаковый исходный массив
`chrono::steady_clock::now()`	Текущее время, точное
`duration<double, milli>`	Переводим разницу во времени в миллисекунды
Почему не `clock()`?	clock() слишком грубый, маленькие сортировки показал бы как 0

runTests — среднее время

```
template <typename T>
void runTests(const char* type, const char* name, void (*fill)(T[], int), int size,
              T src[], T arr[], ofstream& csv) {
    double tIns = 0, tMerge = 0, tStd = 0;
    bool doIns = size <= MAX_N2;

    for (int t = 0; t < TESTS; t++) {
        fill(src, size);
        if (doIns) tIns += measure(insertionSort<T>, src, arr, size);
        tMerge += measure(mergeSort<T>, src, arr, size);
        tStd += measure(stdSort<T>, src, arr, size);
    }
    tIns /= TESTS;
    tMerge /= TESTS;
    tStd /= TESTS;

    cout << setw(10) << size;
    if (doIns) cout << setw(16) << tIns;
    else cout << setw(16) << "-";
    cout << setw(16) << tMerge << setw(16) << tStd << endl;

    csv << type << ";" << name << ";" << size << ";";
    if (doIns) csv << tIns;
    csv << ";" << tMerge << ";" << tStd << endl;
}
```

Что делает?

Считает среднее время трёх сортировок для одного размера массива.

Как делает?

10 раз создаёт новый массив и меряет на нём все три сортировки, складывая время. Потом делит сумму на 10 и печатает строку таблицы на экран и в файл.

Что такое `doIns`?

Нужно ли запускать сортировку вставками. На больших массивах её не запускаем — она слишком долгая.

Что такое `type` и `name`?

Подписи для файла результатов: тип элементов (int или double) и вид данных (random или fragments).

runAll — все замеры для одного типа

```
template <typename T>
void runAll(const char* type, ofstream& csv) {
    int sizes[] = { 1000, 2000, 5000, 10000, 20000, 30000, 40000, 50000,
                    100000, 200000, 500000, 1000000 };
    const int sizeCount = sizeof(sizes) / sizeof(sizes[0]);
    const int maxSize = 1000000;

    T* src = new T[maxSize];
    T* arr = new T[maxSize];

    cout << endl << "Random arrays, " << type << ":" << endl;
    printHeader();
    for (int i = 0; i < sizeCount; i++) {
        runTests(type, "random", fillRandom<T>, sizes[i], src, arr, csv);
    }

    cout << endl << "Arrays with " << PARTS << " sorted fragments, " << type << ":"
            << endl;
    printHeader();
    for (int i = 0; i < sizeCount; i++) {
        runTests(type, "fragments", fillFragments<T>, sizes[i], src, arr, csv);
    }

    delete[] src;
    delete[] arr;
}
```

Что делает?

Делает все замеры для одного типа элементов: `runAll<int>` — для int, `runAll<double>` — для double.

Как делает?

Создаёт через `new` два массива на миллион элементов, проходит по всем размерам сначала со случайными массивами, потом с фрагментами, и в конце удаляет массивы через `delete[]`.

Что такое `sizeof(sizes) / sizeof(sizes[0])`?

Сколько элементов в списке размеров.

Зачем `new`?

Миллион double — 8 МБ, на стек (1 МБ) не поместится.

main

```
int main() {
    srand((unsigned)time(0));

    // Проверка, что шаблоны работают с разными типами
    int a[] = { 2, 5, 7, 8, 1, 4, 3, 9, 6, 0 };
    double b[] = { 3.5, -1.2, 8.0, 0.5, 2.25 };
    insertionSort(a, 10);
    mergeSort(b, 5);
    cout << "insertionSort<int>: ";
    printArray(a, 10);
    cout << "mergeSort<double>:  ";
    printArray(b, 5);
    cout << endl;

    ofstream csv("results.csv");
    csv << "type;data;n;insertionSort;mergeSort;std::sort" << endl;
    cout << fixed << setprecision(3);
    csv << fixed << setprecision(3);

    cout << "Average time, ms (" << TESTS << " arrays for each n)" << endl;

    runAll<int>("int", csv);
    runAll<double>("double", csv);

    cout << endl << "Saved to results.csv" << endl;
    return 0;
}
```

Что делает?

Запускает всё по порядку:

- `srand(time(0))` — чтобы случайные числа были разные при каждом запуске;
- сортирует маленький массив `int` и массив `double` и печатает их — показывает, что шаблоны работают с разными типами;
- открывает файл results.csv и пишет в него заголовки столбцов;
- `fixed << setprecision(3)` — числа с 3 знаками после запятой;
- `runAll<int>` — все замеры на массивах int;
- `runAll<double>` — все замеры на массивах double.

Шаблоны

Что такое `template <typename T>`?

Функция пишется один раз для любого типа. Вместо T компилятор сам подставляет int, double или другой тип.

Почему `insertionSort<T>`, `fillRandom<T>` с угловыми скобками?

Когда функцию передаём как указатель, компилятор не знает, какой тип подставить, поэтому пишем явно.

Схемы (ГОСТ 19.701-90)

Что означают фигуры на схеме?

Овал — начало и конец. Прямоугольник — действие. Ромб — условие, из него два выхода «да» и «нет». Фигура со срезанными углами (как трапеция) — граница цикла for: сверху начало цикла с условием, снизу конец цикла, тело между ними. Цикл while нарисован ромбом со стрелкой назад к условию. Прямоугольник с двойными боковыми линиями — вызов функции (у нас рекурсивный вызов mergeSortRange).

«Почему так, а не иначе»

Почему `j >= 0` первым в while?

Чтобы при j = −1 не обратиться к arr[−1]: оператор && дальше не проверяет.

Почему середина `(right - left) / 2 + left`?

Так нет переполнения int на больших числах.

Почему `<=` в слиянии?

Равные элементы остаются в исходном порядке (сортировка устойчивая).